

Lifelikeygame Technical Development Plan

1. 保留技术栈

- Frontend : React + TypeScript
- Backend : FastAPI
- Database : PostgreSQL
- ORM : SQLAlchemy
- Migration : Alembic
- Authentication : JWT + Google Login
- AI : 可切换 AI Provider
- Local AI : Ollama
- Testing : Pytest
- Deployment : Docker

2. Backend 架构目标

采用：

FastAPI + Feature Folder + Hybrid CQRS + Lightweight Mediator + Unit of Work + Pipeline Behavior

简单 CRUD

```text id="jd8wou" FastAPI Router → Application Service → Repository → PostgreSQL

适用功能：

- Get current user
- List goals
- Get goal by ID
- Update goal title
- List rewards
- List scoring schemes
- 简单单表查询
- 简单单表更新

## 复杂业务流程

```text id="5n1yth"

FastAPI Router

→ Mediator.send(Command)

- Pipeline Behaviors
- Command Handler
- Repository / Domain Service
- Unit of Work
- PostgreSQL

适用功能：

- CreateTaskWithScheduleCommand
- CompleteTaskInstanceCommand
- RedeemRewardCommand
- ConfirmAiPlanCommand
- GenerateDailyTaskInstancesCommand
- LoginWithGoogleCommand
- LinkGoogleAccountCommand

3. Command、Query 与 Handler

Command

用于改变系统状态。

```
```text id="xgfv87" CreateTaskWithScheduleCommand CompleteTaskInstanceCommand
RedeemRewardCommand ConfirmAiPlanCommand GenerateDailyTaskInstancesCommand
LoginWithGoogleCommand LinkGoogleAccountCommand
```

## Query

用于读取资料。

```
```text id="h0jmhc"
GetGoalByIdQuery
ListGoalsQuery
ListTodayTaskInstancesQuery
GetPointBalanceQuery
GetDashboardQuery
```

初期只要求复杂写入使用 Command。

简单 Query 可以继续通过 Service 处理。

Handler 职责

Handler 负责：

- 执行一个明确 Use Case
- 验证业务条件
- 验证资源 Ownership
- 控制执行顺序
- 调用 Repository
- 调用共用业务 Service
- 返回 Result DTO

Handler 不负责：

- 解析 HTTP Request
- 使用 FastAPI Request Schema
- 直接返回 HTTPException
- 构造 API Response Schema
- 自行执行分散的 Commit

4. Lightweight Mediator

Mediator 负责：

```text id="zm86g1" Request Type → 找到对应 Handler → 执行 Pipeline → Handler.handle() → 返回结果

第一版功能：

- Handler Registry
- `send(request)`
- Request 与 Handler 映射
- Duplicate Handler 检查
- Handler Not Found 错误
- Pipeline 执行
- Command 与 Handler 日志

Handler 注册方式：

```
```python id="smy3hb"
@handles(CompleteTaskInstanceCommand)
class CompleteTaskInstanceCommandHandler:
    ...
```

初期不加入：

- Event Bus
- Saga
- Distributed Messaging
- 复杂自动扫描
- 过度 Reflection

5. Pipeline Behavior

计划加入：

```text id="5q9dtd" Logging Behavior Validation Behavior Transaction Behavior Performance Behavior

执行流程：

```
```text id="p42k35"
Mediator.send()
  → Logging
  → Validation
  → Transaction
  → Handler
  → Commit
```

异常流程：

```text id="mp8hrk" Handler Error → Rollback → Logging → Domain Exception → FastAPI Global Exception Handler

## Logging Behavior

记录：

- Request ID
- Command Type
- Handler Type
- User ID
- Execution Time
- Success / Failure
- Transaction Result

不记录：

- Password
- Password Hash
- JWT
- Google ID Token
- API Key
- 完整敏感 Payload

## ## Validation Behavior

负责：

- Command 字段验证
- 字符长度
- 日期范围
- Task 数量
- Completion Level
- AI Plan 限制
- 必填资料检查

## ## Transaction Behavior

负责：

```
```text id="o3eq7e"
Begin
→ Handler
→ Commit
```

异常时：

```
```text id="w5xy91" Rollback → Raise Error
```

```

```

## # 6. Repository 与 Unit of Work

### ## Repository

保留具体 Repository：

```
```text id="bsnwmn"
UserRepository
UserIdentityRepository
GoalRepository
```

```
TaskRepository
TaskScheduleRepository
TaskInstanceRepository
ScoringSchemeRepository
PointLedgerRepository
RewardRepository
RedemptionRepository
```

Repository 负责：

- add
- get
- list
- query
- delete
- flush
- refresh

Repository 不负责：

- commit
- rollback
- 跨模块业务流程
- Ownership 判断
- 积分计算
- HTTP Response

Unit of Work

Unit of Work 负责：

- begin
- commit
- rollback
- transaction boundary

规则：

- Repository 不自行 Commit
- 一个 Use Case 只有一个 Transaction Owner
- 简单写入由 Application Service 控制 Transaction
- 复杂 Command 由 Transaction Behavior 和 Unit of Work 控制

7. DTO、Schema 与 Entity

Request 流程

``text id="hdr6rk" Frontend JSON → FastAPI Request Schema → Router Mapping → Command / DTO → Handler / Service → Entity → Repository

Response 流程

```
``text id="mykkq5"
Entity / Query Result
  → Result DTO
  → Router Mapping
  → Response Schema
  → Frontend JSON
```

规则：

- Router 使用 Request / Response Schema
- Handler 不引用 API Schema
- Service 不引用 API Schema
- Handler 与 Service 不执行 `model_dump()`
- Repository 接收 Entity 或明确查询参数
- Command 和 DTO 尽量不可变
- Update DTO 区分：
 - 字段未提供
 - 字段明确设为 null

8. Application Service 与业务 Policy

适合保留或新增的共用 Service：

``text id="ur1ff5" TokenService GoogleIdentityService AIPlannerProvider AIPlanValidationService TaskCompletionScoringPolicy ScheduleMatchingService PointBalanceService OwnershipService

职责：

- 多个 Handler 共用逻辑
- 外部服务整合
- 业务计算
- 规则验证

- Token 创建与验证
- AI Provider 调用

完整业务 Use Case 应放在 Handler, 不放进大型 Service。

9. 错误处理

成功：

```
```text id="t6y8ed"
Handler
→ 返回 Result DTO
```

业务失败：

```text id="enw9jn" Handler / Service → 抛出 Domain Exception

HTTP 转换：

```
```text id="2v7lbc"
Global Exception Handler
→ 统一 HTTP Error
```

计划新增：

```text id="ikc08z" InsufficientPointsError InvalidCompletionLevelError RewardAlreadyRedeemedError  
TaskInstanceAlreadyExistsError ResourceNotOwnedError AccountLinkRequiredError
GoogleTokenInvalidError AIProviderTimeoutError AIProviderUnavailableError AIPlanValidationError

统一错误格式：

```
```json id="5zy8u4"
{
 "error": {
 "code": "INSUFFICIENT_POINTS",
 "message": "Not enough points.",
 "details": {
 "required": 100,
 "available": 60
 }
 }
}
```



```
}
}
```

## 10. Backend 目录结构

```
``text id="mkegg5" backend/app/ ├── core/ | ├── config.py | ├── mediator.py | ├──
unit_of_work.py | ├── behaviors.py | ├── errors.py | └── logging.py | ├── auth/ | ├──
commands.py | ├── command_handlers.py | ├── schemas.py | ├── router.py | ├──
google_identity_service.py | └── token_service.py | ├── users/ | ├── models.py | ├──
repository.py | ├── service.py | ├── interfaces.py | └── schemas.py | ├── tasks/ | ├──
commands.py | ├── command_handlers.py | ├── queries.py | ├── query_handlers.py | ├──
dtos.py | ├── models.py | ├── repository.py | ├── service.py | └── router.py | ├──
task_instances/ | ├── commands.py | ├── command_handlers.py | ├── queries.py | ├──
query_handlers.py | ├── policies.py | ├── models.py | ├── repository.py | └── router.py |
└── goals/ ├── task_schedules/ ├── scoring_schemes/ ├── point_ledgers/ ├── rewards/ ├──
redemptions/ ├── ai_planner/ └── scheduler/
```

初期使用：

```
```text id="s4nm13"  
commands.py  
command_handlers.py  
queries.py  
query_handlers.py
```

文件变大后再拆成：

```
``text id="pffxus" complete_task/ redeem_reward/ confirm_ai_plan/
```

```
---
```

```
# 11. Google 登录
```

```
## 登录流程
```

```
```text id="qbb2x7"
```

```
React
```

- Google Identity Services
- Google ID Token
- POST /auth/google
- FastAPI 验证 Google Token

- 找到或建立本地 User
- 建立 User Identity
- 签发 Lifelikegame JWT

后续 API 继续使用 Lifelikegame JWT。

## User Identity 表

```
```text id="zz1bkd" user_identities |—— id |—— user_id |—— provider |—— provider_subject |——
provider_email |—— created_at |—— updated_at
```

数据库约束：

```
```text id="7qu0bp"
UNIQUE(provider, provider_subject)
```

## Google Command

```
```text id="u18gt5" LoginWithGoogleCommand → LoginWithGoogleCommandHandler
```

未来加入：

```
```text id="255kjk"
LinkGoogleAccountCommand
UnlinkGoogleAccountCommand
```

## Account Linking

### Identity 已存在

```
```text id="jhkj7u" 直接登录对应 User
```

Identity 不存在, Email 不存在

```
```text id="nw4j6k"
建立 User
建立 User Identity
签发 JWT
```

## Identity 不存在，但 Email 已存在

```text id="azhsfv" 返回 ACCOUNT\_LINK\_REQUIRED 要求用户先通过原登录方式登录 再主动绑定 Google

```
## Password

Google-only 用户 :

```text id="0gl2gp"
password_hash = null
```

不建立随机密码。

---

## 12. AI Provider

### 环境安排

```text id="qdugqj" Development → 本机 Ollama

Production 初期 → 便宜的云端模型

未来高流量 → 再评估 GPU 自托管

```
## Ollama 定位

- 本地开发
- Prompt 调整
- Structured Output 测试
- 自动化测试
- 本地模型质量比较
- 未来自托管候选

## AI Provider Interface

```python id="v6lnag"
class AIPlannerProvider(Protocol):
 async def generate_plan(
 self,
 request: PlanGenerationRequest,
) -> GeneratedPlan:
 ...
```

实现：

```
```text id="bzlfq5" GeminiPlannerProvider CloudPlannerProvider OllamaPlannerProvider  
FallbackAIPlannerProvider
```

配置：

```
```env id="qut8kl"  
AI_PROVIDER=...
AI_MODEL=...
AI_TIMEOUT_SECONDS=...
AI_MAX_RETRIES=...
```

Ollama：

```
```env id="6tnipx" OLLAMA_BASE_URL=http://localhost:11434 OLLAMA_MODEL=...
```

Fallback 条件

允许 Fallback：

- Timeout
- Connection Failure
- 429
- Temporary 5xx

不允许 Fallback：

- 用户资料不足
- Ownership 失败
- 业务验证失败
- 计划数量超限
- 日期无效
- 用户输入无效

AI 成本控制

代码负责：

- 检查缺失字段
- 日期验证
- Task 数量限制
- Schedule 完整性
- JSON Schema 验证

- 重复 Task 检查
- Scoring Scheme Ownership
- AI 使用次数限制

AI 负责：

- 理解自然语言目标
- 拆分任务
- 生成简短说明
- 提出合理频率与安排

限制：

- 每次最多生成 5-8 个初始 Task
- 限制 Prompt 长度
- 限制输出长度
- 限制 Conversation History
- 限制每个用户每日完整计划生成次数
- 防止重复点击
- 记录每次调用 Token 与成本

AI Benchmark

固定测试场景：

```text id="5xzq23"

减重计划

学习日语

准备考试

改善睡眠

储蓄目标

每天只有 15 分钟

周末才有时间

中文输入

英文输入

中英混合输入

资料不足需要追问

记录：

- First Token Latency
- Total Latency
- Schema Valid Rate
- 计划合理性
- Constraint Adherence
- Retry Rate

- Failure Rate
- 单次费用
- 中文质量

---

## 13. Complete Task 资料正确性

以下操作必须属于同一 Transaction：

```text id="w0x586" 更新 Task Instance 建立或更新 Point Ledger 更新 User Balance

需要处理：

- `delta == 0` 不新增 Ledger
- 重复请求幂等
- 并发保护
- 相同完成等级不会重复加分
- 修改完成等级只计算差额
- Ledger 与 User Balance 保持一致

14. Reward Redemption 资料正确性

以下操作必须属于同一 Transaction：

```text id="9xucdw"

检查 Reward

检查余额

扣除余额

更新 Reward 状态

建立 Redemption

建立负数 Ledger

需要处理：

- Reward 重复兑换
- 并发兑换
- User Balance 不可错误变负
- 一次性 Reward Unique Constraint
- 失败时全部 Rollback
- Response 使用更新后的 User Balance

## 15. Task Instance 资料正确性

增加数据库 Unique Constraint。

需要决定：

```
```text id="f80q3g" UNIQUE(task_id, date_instance)
```

或：

```
```text id="2v86ue"
UNIQUE(task_schedule_id, date_instance)
```

同时加入：

- Duplicate Error Handling
- Scheduler 重复执行测试
- 并发创建测试
- Soft Delete 过滤
- Active 状态过滤

---

## 16. Point Source of Truth

建议：

```
```text id="72zcd7" Point Ledger → Source of Truth
```

User.current_value → 缓存余额

要求：

- Ledger 与 Balance 同一 Transaction 更新
- 定期执行 Reconciliation
- 验证：

```
```text id="xp1d8e"
User.current_value
=
SUM(PointLedger.delta)
```

## 17. Scheduler

需要完成：

- 异常 Rollback
- Structured Logging
- Active Task Filter
- Active Schedule Filter
- Soft Delete Filter
- Duplicate Generation Protection
- Monthly 29/30/31 规则
- Scheduler 与 API Process 分离

生产结构：

```text id="sf2ksy" API Container Scheduler Worker PostgreSQL

短期配置：

```
```env id="4gx4k4"  
SCHEDULER_ENABLED=false
```

只允许一个 Process 启动 Scheduler。

---

## 18. Soft Delete 与 Ownership

### Soft Delete

统一规则：

- 所有普通 Query 默认过滤 `deleted_at`
- 已删除资料不可 Update
- 已删除资料不可生成 Task Instance
- 已删除资料不可兑换
- 管理员查询另行处理
- Restore 行为明确

### Ownership

所有资源操作验证：

```text id="op81y6" User A 建立资源 User B 尝试读取 User B 尝试更新 User B 尝试删除 User B 尝试执行业务动作


必须覆盖：

- Goals
- Tasks
- Task Schedules
- Task Instances
- Rewards
- Redemptions
- Scoring Schemes
- Point Ledgers
- AI Drafts
- User Identities

19. 实际执行顺序

Phase 1：简单安全修正

1. 删除密码 Hash Debug 输出
2. 删除 JWT `dev-secret`
3. 隐藏 `/db-health` 数据库版本
4. Redemption Response 使用 Updated User
5. `delta == 0` 不新增 Ledger
6. 使用 Logging 取代 Print
7. 建立 Typed Settings
8. 建立 Domain Error

Phase 2：接口与模块边界

1. 修正 Interface 参数类型
2. 修正 Interface Return Type
3. Task 模块建立 DTO
4. Router 负责 Schema → DTO
5. TaskService 移除 Schema Import
6. TaskService 移除 `model\_dump()`
7. Service 不调用其他模块 Repository
8. Ownership 改用 Service Interface
9. 逐模块迁移 DTO

Phase 3：Transaction 基础

1. 定义 Transaction Ownership
2. 建立 Unit of Work
3. Repository 移除自动 Commit
4. Task + Schedule + Instance 原子化

5. Complete Task 原子化
6. Redemption 原子化
7. AI Plan 原子化

Phase 4 : Hybrid CQRS

1. 建立 Command Base
2. 建立 Query Base
3. 建立 Handler Protocol
4. 建立 Lightweight Mediator
5. 建立 Handler Registry
6. 建立 Transaction Behavior
7. 建立 Logging Behavior
8. 建立 Validation Behavior
9. 建立 Performance Behavior

第一批迁移：

```
```text id="dq3mq7"
CompleteTaskInstanceCommand
RedeemRewardCommand
CreateTaskWithScheduleCommand
```

第二批迁移：

```
text id="c4jbba"
ConfirmAiPlanCommand
GenerateDailyTaskInstancesCommand
```

## Phase 5 : Google 登录

1. 抽出 TokenService
2. 新增 UserIdentity Model
3. 新增 UserIdentity Repository
4. 新增 Alembic Migration
5. 新增 GoogleIdentityService
6. 新增 LoginWithGoogleCommand
7. 新增 LoginWithGoogleCommandHandler
8. 新增 `/auth/google`
9. React 加入 Google Login Button
10. 实现 Account Linking
11. 实现 Unlink Google
12. 加入 Google 登录测试

## Phase 6 : AI Provider

1. 建立 AIPlannerProvider Interface
2. 将现有 Gemini 包装为 Provider
3. 使用 Structured Output
4. 加入 Timeout
5. 加入 Retry
6. 加入 AI Error Classification
7. 减少 Prompt
8. 减少 Conversation History
9. 加入用户使用次数限制
10. 加入 Token 和成本记录
11. 加入 OllamaProvider
12. 本机测试本地模型
13. 加入第二个云端 Provider
14. 建立 Fallback Provider
15. 建立 AI Benchmark
16. 根据测试结果选择 Production Provider

## Phase 7 : 资料完整性

1. Task Instance Unique Constraint
2. Ledger Idempotency
3. Complete Task 并发控制
4. Redemption 并发控制
5. 明确 Point Source of Truth
6. 建立 Point Reconciliation
7. Soft Delete 全面审计
8. Ownership 全面审计
9. FK 与 Delete Behavior 审计
10. Database Check Constraint 审计

## Phase 8 : Scheduler 与 AI Workflow

1. Scheduler Rollback
2. Scheduler Structured Logging
3. Scheduler Active Filter
4. Scheduler Soft Delete Filter
5. Scheduler Duplicate Protection
6. Scheduler 独立 Worker
7. Monthly Schedule 规则
8. AI Draft 保存到 Backend
9. AI Confirm 只接受 Draft ID
10. AI Plan Business Limits
11. AI Plan Ownership 验证
12. AI Draft 重复确认保护

## Phase 9 : Production Gate

上线前完成：

- Transaction Rollback Integration Tests
- Concurrency Tests
- Idempotency Tests
- Ownership Test Matrix
- Soft Delete Tests
- Scheduler Date Boundary Tests
- Scheduler Duplicate Tests
- Google Token Verification Tests
- Google Account Linking Tests
- AI Timeout Tests
- AI Retry Tests
- AI Invalid Output Tests
- Dependency Version Lock
- Backend CI
- Alembic Migration Deployment Flow
- Login Rate Limiting
- Signup Rate Limiting
- AI Rate Limiting
- Request ID
- Error Monitoring
- Liveness Endpoint
- Readiness Endpoint
- Production Database Connection Pool
- Secret Environment Management
- Backup 与 Restore 测试